

Unofficial solution outlines for the three requested tasks. The presentation is independently typeset from the official statements and public post-contest solution notes.

Problem A. Make All Equal

At every moment the piles are sorted by height. The key is to work in levels. For $p = 2, 4, 8, \dots, N$, assume inductively that every consecutive block of $p/2$ piles has one common height. Treat each such block as a single *super-pile*; consecutive super-piles are paired to form blocks of size p .

A halving lemma

Suppose the two values in every adjacent pair differ by at most $2X$. Add X to the lower value of every pair, and sort all values again. After sorting, every new adjacent pair differs by at most X .

Indeed, before sorting, match each raised lower value with its former partner. If the old gap is $d \leq 2X$, the new gap in this matching is $|d - X| \leq X$. On a line, pairing consecutive values after sorting cannot have a larger bottleneck than an existing perfect matching. Hence every consecutive pair in the sorted order has gap at most X .

The same argument applies to super-piles: all $p/2$ equal copies are always modified together. Sorting may permute the super-piles, but ties are harmless, and the sorted multiset can again be split into consecutive groups of $p/2$ equal values.

One level

Let $p = 2^\ell$. For each

$$X = 2^{18+\ell}, 2^{17+\ell}, \dots, 2, 1,$$

call `add` on the first half of every current block of length p . At the start of this level the total range of all heights is less than $2^{19+\ell} = 2 \cdot 2^{18+\ell}$: initially it is below 2^{20} , and level $j < \ell$ can increase the range by at most 2^{19+j} . The halving lemma therefore applies repeatedly. After the call with $X = 1$, the two super-piles in every block differ by at most 1.

Compare the first and last pile of each block. If they differ, the first half is exactly one lower than the second half; collect the first halves of all such blocks and add 1 to them in one final call. Every block of length p is now constant, establishing the invariant for the next level.

For $k = \log_2 N$, the number of comparison calls is

$$\frac{N}{2} + \frac{N}{4} + \dots + 1 = N - 1.$$

The number of addition calls is at most

$$\sum_{\ell=1}^k (20 + \ell) = 20k + \frac{k(k+1)}{2}.$$

For $N \leq 2048$ ($k \leq 11$), these bounds are 2047 comparisons and 286 additions, within the full-score limits. The running time needed to prepare the index vectors is $O(N \log N)$.

Problem B. Mazes

Use the following 3×3 gadget, where dots are empty cells and the centre is blocked:

```
. . .  
. # .  
. . .
```

There are exactly two monotone paths from its top-left cell to its bottom-right cell: one passes above and to the right of the centre, and the other passes to the left and below it.

Place these gadgets along the main diagonal so that the bottom-right cell of one gadget is the top-left cell of the next. Consequently, exactly 2^p paths reach the entrance of the gadget with index p (indices start at 0).

Write

$$K = \sum_{p \geq 0} b_p 2^p, \quad b_p \in \{0, 1\}.$$

For every set bit $b_p = 1$, open a vertical corridor from the entrance of gadget p down to the last row. Make the last row completely empty, so that this corridor has a unique continuation to the bottom-right exit. The corridor shares the left side of its gadget, so it does not create an additional way to continue along the diagonal chain; it only adds the option of leaving the chain.

Thus the corridor belonging to bit p contributes exactly 2^p complete routes. Stop the diagonal construction after the highest set bit, leaving the unfinished diagonal branch disconnected from the exit. The total number of routes is therefore

$$\sum_{p: b_p=1} 2^p = K.$$

Since $K \leq 10^{18} < 2^{60}$, fewer than 60 gadgets are needed. Their diagonal extent is below 120, so a fixed 200×200 grid is sufficient. The construction takes $O(200^2 + 200 \log K)$ time, dominated by writing the output grid.

Problem C. Parcel Post

Consider a directed path segment from station s to station t . A high-power firing has a one-time setup cost B_s and then costs C for every traversed edge. Therefore the only information that must cross the boundary between two consecutive path segments is whether a high-power firing has already been paid for and may continue.

For each directed segment, store a 2×2 min-cost matrix M . Row i describes the state at the start and column j the state at the end:

- state 0 (closed): no previously opened high-power firing is available;
- state 1 (open): a previously paid high-power firing may be continued, at cost C per additional edge.

Leaving a segment in state 1 means that the setup cost of a firing which can continue past the endpoint has already been included.

For one directed edge $u \rightarrow v$, the matrix is

$$M_{u \rightarrow v} = \begin{pmatrix} \min(A_u, B_u + C) & \min(A_u + B_v, B_u + C) \\ \min(C, A_u) & \min(C, A_u + B_v) \end{pmatrix}.$$

For example, the upper-right entry either sends the edge at low power and opens a new high-power firing at v , or starts a high-power firing at u and keeps it open after crossing the edge.

If segment P is followed by segment Q , their matrices combine by min-plus multiplication:

$$(M_P \otimes M_Q)[i][k] = \min_{j \in \{0,1\}} (M_P[i][j] + M_Q[j][k]).$$

This is exactly the dynamic-programming transition at their common endpoint. Matrix multiplication is not commutative, so path direction must be respected.

Root the tree arbitrarily and build binary-lifting tables. For every vertex v and exponent r , store

- $\text{up}[r][v]$: the matrix for the directed path from v to its 2^r -th ancestor;
- $\text{down}[r][v]$: the matrix for the reverse directed path, from that ancestor to v .

The recurrences are

$$\begin{aligned} \text{up}[r][v] &= \text{up}[r-1][v] \otimes \text{up}[r-1][\text{anc}_{r-1}(v)], \\ \text{down}[r][v] &= \text{down}[r-1][\text{anc}_{r-1}(v)] \otimes \text{down}[r-1][v]. \end{aligned}$$

For a query (X, Y) , let $L = \text{LCA}(X, Y)$. Multiply the required lifting blocks for $X \rightarrow L$ in upward order. While lifting Y to L , save the downward blocks and multiply them in reverse order to obtain $L \rightarrow Y$. Concatenate the two products. A complete parcel route starts and ends closed, so the answer is the $[0][0]$ entry of the resulting matrix.

Preprocessing uses $O(N \log N)$ time and memory. Each query takes $O(\log N)$ time, for total complexity $O((N + Q) \log N)$.

Sources and status

This is not an official WEOI editorial. It is an independent solution-outline compilation based on the official WEOI 2024 task statements and Suneet Mahajan's public post-contest editorial.

Official contest page: weoi.org/weoi-2024. Public solution notes: Suneet Mahajan, "WEOI 2024 Editorial".